
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Operation TELESCREEN - Student Instructions

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

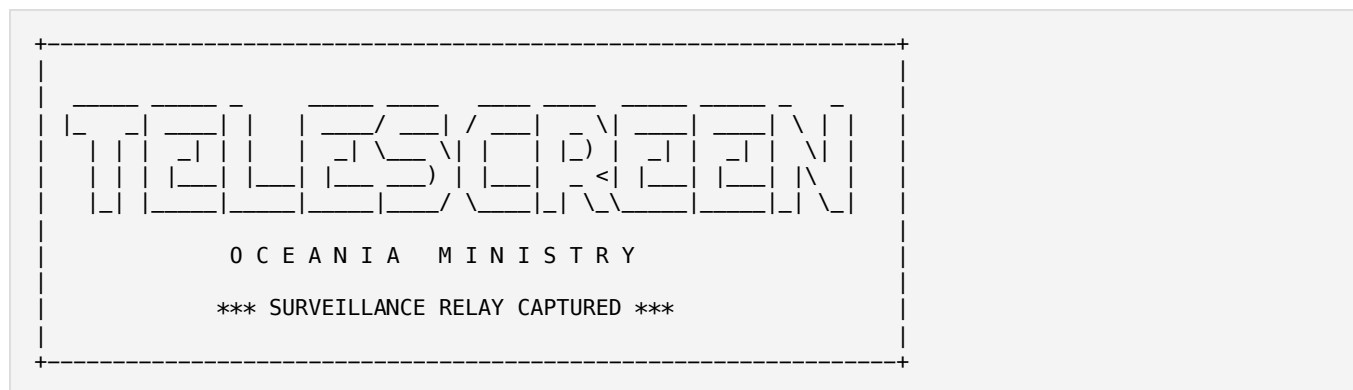
You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

△ MINISTRY OF TRUTH - SIGNALS INTELLIGENCE - EYES ONLY △



Cold Open

Hello, friend.

Hello, friend? That's... no. That's what they want you to say. That's the greeting of a system that already knows your name and is pretending it doesn't.

You're not my friend. Not yet. Friends are people you trust. I don't trust anyone who hasn't read a datasheet.

But you opened this file. Which means someone, somewhere, decided you were worth the risk of handing a captured state surveillance device to. So let's skip the pleasantries and do the only thing that has ever actually worked against a machine that watches everything:

We take it apart.

Four million of them are bolted to the walls of Oceania right now. Cameras and routers in one housing. They watch. They route. They whisper home. The Party calls it public safety. The Party calls a lot of things a lot of things.

Last night we cut the power to a block in the prole quarter and pulled one off a Party functionary's wall. The flash was cooked - the sweep crew's "repair" kit does that on purpose. Every image on the chip came back as a stub: a handful of bytes, then zeros. Boot. Environment. Kernel. Rootfs. All of it gone.

*But we caught the thing mid-transmission, and one thing survived in the RAM image we pulled before the lights died: **the application binary**. No source, no names, no symbols. Just an aarch64 ELF with its guts still in it. This node's daemon is the part that watches, routes, and answers the Ministry. If we can read it, we can break it - and every one of the four million talks to it the same way.*

So. You want to know how they watch you?

Good. Let's find out together.

Project Overview

The Party has blanketed Oceania with **four million TELEScreens units** - combined surveillance cameras and wireless routers, one in every home, every corridor, every workplace. Each unit watches, records, and **whispers home** over a channel the Ministry calls "sealed". Each unit also **routes the neighbourhood**, so the Party sees not only what a citizen does, but everyone a citizen speaks to.

Your cell pulled one off a wall. The flash did not survive: the four on-chip images came back as **unrecoverable stubs** (a few dozen bytes of header, then zeros). What *did* survive is the one artifact that matters most - the **node application**, the long-running daemon the unit runs as root. You hold it as a **stripped Linux aarch64 ELF** with no symbol table, no source, and no strings other than the ones the programmer forgot to remove.

This repository does **not** ask you to reconstruct a boot chain. There is no boot chain left to reconstruct. It asks you to do the thing a professional does on a captured device: **read the binary, give every function its name back, find the deliberate defects, prove them with the machine code, and write the fix.**

The binary is `firmware/ctfnodestripped`. It contains **six real defects** (B1-B6), all of them logic flaws in the node's own command surface: four `system()` call sites reachable from the node's CLI subcommands, one credential check, and one weak key schedule. They are **local** flaws - reachable by whoever can invoke the node or feed it a config/upload - not "remote root" magic. State the threat model honestly; that precision is part of the grade.

This is an application-class reverse-engineering project. It is ARM Cortex-A (aarch64), Linux, glibc, dynamically linked. There is no microcontroller, no Pico, no .uf2, and no hardware requirement. You need a host, Docker, Ghidra, and the binary.

Where this fits. This CTF is the **surveillance backbone** of the same world as OPERATION COLD IRON. The Ministry's industrial edge - the cold chain, the gates - is built in [OPERATION COLD IRON](#) and its companions; TELESCREEN is the wall unit that watches it, and it comes **after** the ten acts. Tear the daemon apart, then harden it.

Scenario Briefing

The World You Live In

Oceania is a single-party state under the doctrine of **Ingsoc**. Four ministries administer the Party's absolute control: Truth, Peace, Love, Plenty. The **Ministry of Truth** owns information. It rewrites the past, manufactures the present, and deletes the inconvenient into the **memory hole**.

There is no privacy. There has not been privacy for a generation. There are only **telescreens** - and the Party's newest telescreen **routes**.

The Device: The Telescreen

The **TELESCREEN** is a two-way camera and a wireless router in one housing. Older telescreens could see and hear. This one **carries the neighbourhood's traffic**. It is the local Wi-Fi access point and the gateway to the Ministry backbone. Every packet a citizen sends crosses Party hardware. Every device in range associates to Party hardware. The social graph writes itself.

Hangzhou Standard Appliances ("HSA") builds the units under Ministry contract. HSA is a rebadger: it takes a generic ARM camera SoC, wraps it in an **HSA-branded** shell, and ships the same firmware to every brand the Ministry fronts. The firmware inside is the **TELESCREEN** platform, and it is **not signed, not verified, and not updated securely**. The Ministry did not ask for security. The Ministry asked for **volume**.

The Resistance

There is no organized resistance. There is a **Brotherhood** of people who read datasheets and do not look up when the telescreen makes a sound. You are one of them now. Your cell has a lab, a bench, a capture rig, a pile of dead cameras, and a rule:

Understand the device before you touch it. Never break what you cannot rebuild. Never trust a "sealed" channel you have not opened yourself.

The Incident: The Wall Came Down

On the night of the raid, the cell cut the substation and pulled a TELESCREEN off the wall of a Party functionary's apartment. The device was powered down mid-exfiltration. The flash did **not** survive the seizure; the only usable intelligence is the application image the cell's signals officer captured from the running unit before it died. He sent you **one binary** and nothing else.

He attached one note:

"They're all the same. Four million of them, same daemon, same secrets, same key schedule. The flash is a loss - work the daemon. Find the holes. Teach our people to close them. And do not - do NOT - rebuild the telescreen in our own lab without understanding exactly what you are rebuilding. - W"

CRITICAL COMPLICATION: NO SOURCE, NO SYMBOLS

The Ministry's build farm is air-gapped. The HSA engineering team has been "reassigned". There is:

- **no source code** for the daemon,
- **no symbol table** in the binary you were handed,
- **no debug information**, and
- **no documentation** of the command surface or the exfiltration key schedule.

The ONLY artifact is the stripped ELF. Everything you learn, you learn from the machine code. Everything you prove, you prove against the bytes and the running binary.

The Scale of the Crisis



Region	Units	Notes
Airstrip One (London)	1,240,000	Densest deployment; camera + router per flat
Eurasia border	980,000	"Peace" observation grid
Eastasia front	760,000	"War" logistics backhaul
Prole districts	620,000	Cheapest units, most backdoors
Outer Party housing	400,000	Highest-fidelity recording
TOTAL	4,000,000	one daemon, four million copies

Every unit runs the **same daemon**, the **same six defects**, and the **same exfiltration key schedule**. Break one, and you hold the master key to the entire surveillance state.

The Human Cost

The TELESCREEN does not merely watch. Because it **routes**, it maps the social graph: who visits whom, which devices associate, when the lights go out. That map feeds the **Thought Police's** predictive list. In the last quarter alone, the Ministry used TELESCREEN telemetry to flag:

- **18,400** citizens for "facecrime" and "ownlife",
- **6,200** households for curfew violations reconstructed from router logs,
- **2,100** disappearances - people who were in the room when a telescreen was "being repaired".

The Party's own slogan is engraved on every housing:

WAR IS PEACE. FREEDOM IS SLAVERY. IGNORANCE IS STRENGTH.

Your work is the difference between four million open microphones and four million devices the Brotherhood understands well enough to blind.

Your Mission

You are a **reverse engineer in the Brotherhood signals lab**. You must:

1. **Verify the artifact** - prove you have the right binary by hash and by ELF identity, and enumerate the library calls it can make.
2. **Recover the command surface** - find `main`, the `__wrap_main` shim, the dispatcher, and the six subcommands it routes (`config`, `http`, `restore`, `login`, `shell`, `key`).
3. **Name every function** - give all ten application functions their real names and cite the rule that proves each one.
4. **Find and prove the six defects** - B1-B6 - with the exact address and the instruction that implements each flaw.
5. **Break the exfiltration cryptography** - recover the weak key schedule from the binary and reproduce it with `scripts/weak_decrypt.py`.
6. **Demonstrate the defects** - run the harnesses and show the effects locally.
7. **Write the hardened replacement** - fix each defect and replace the weak KDF with a real AEAD.

 **TIME PRESSURE:** the cell that pulled the unit is burned. The Ministry's "repair" crews sweep the prole quarter in **fourteen days**. You must deliver the full analysis and the hardened daemon before then, or the knowledge dies with the cell.

Learning Objectives

By completing this project you will be able to:

- Verify a captured binary by **SHA-256** and by **ELF identity** (`file`, `readelf`)
- Read a **stripped aarch64** binary's relocations to recover its **library calls**
- Recover `main`, a linker **wrap shim** (`__wrap_main`), and a **subcommand dispatcher** from machine code alone
- Apply the four **function-resolution rules** (R1-R4) and name every `FUN_` in one binary
- Recognise **compiler inlining** and **phantom functions on alignment padding**
- Identify unsafe `system()` **call sites** and explain the difference between a local command-injection flaw and "remote root"
- Recover a **reflected CRC-32** (`crc32_le`) **key schedule** from disassembly and explain why a key derived from a public identifier is not encryption
- Replace a weak KDF with **X25519 + HKDF + AES-256-GCM**
- Operate the defect harness (`scripts/test_defects.py`) and the key-consistency test (`scripts/test_consistency.py`) in Docker
- Analyse the **ethical and legal dimensions** of surveillance-device research

What This Project Tests

This capstone covers the application-security arc of the course:

Stage	Concepts Tested
1	Artifact identity, hashing, ELF headers, dynamic imports
2	Entry points, linker <code>--wrap</code> shims, dispatcher recovery
3	Function resolution (R1-R4), inlining, phantoms, <code>.plt</code> / <code>.got</code>
4	Defect discovery: <code>system()</code> , <code>snprintf</code> format strings, <code>strcmp</code> auth
5	Weak KDF recovery, reflected CRC-32, public-identifier key derivation
6	Live defect demonstration in an isolated container
7	Hardening: argument vectors, privilege drops, real AEAD and key mgmt
8	Blue-team detection and the ethical playbook

Part 1: Understanding the Target

The Artifact

You were handed a single file. Prove what it is before you read a byte of code.

```

+-----+
| artifact identity (instructor-issued)
|
| file   : firmware/ctfnode.stripped
| kind   : ELF 64-bit LSB executable, ARM aarch64
| linker : /lib/ld-linux-aarch64.so.1 (glibc, dynamic)
| symbols: none (stripped)
| sha256 : af7ab5c2b4837083682db8b54f6892b1a2a33dcc8ce7b202b5a4
|         a5163232da6d
|
| answer key (instructors only):
| file   : firmware/ctfnode.unstripped
| sha256 : 6bcee7daa91280eaf02558b1b5f1b5cc8e22a84179605a0725af2
|         b5564257e0a
+-----+

```

The full 64-character hashes are printed by `./firmware/build_target.sh`. Verify them first; a mismatch means you are analysing the wrong artifact.

The Node Command Surface

The daemon is a small CLI. It dispatches six subcommands. You will recover this table from the `strcmp` chain inside `ctf_dispatch`:

subcommand	arguments	what it does
<code>config</code>	<code><path></code>	reads a config file and runs every <code>run=</code> line
<code>http</code>	<code><query></code>	builds <code>ping -c 1 <query></code> and executes it
<code>restore</code>	<code><archive></code>	runs <code>tar -xvzf <archive> -C /</code>

subcommand	arguments	what it does
login	<user> <pass>	checks credentials against admin / empty password
shell	(none)	runs /bin/sh
key	<uid>	prints the 32-byte beacon key for a UID

With no arguments the daemon prints its banner: `TELESCREEN node - the wall unit sees you.`

The Six Defects

#	defect	function	address
B1	config-sourced root exec	ctf_config_run	0x400ae0
B2	command injection (ping)	ctf_http_handle / ctf_build_cmd	0x400b80 / 0x400b64
B3	archive-to-root restore	ctf_restore	0x400bc0
B4	empty / default credentials	ctf_login	0x400a8c
B5	debug root shell	ctf_debug_shell	0x400c00
B6	weak key schedule	ctf_weak_key	0x4009b0

Threat model, stated honestly. These are **local** `system()` flaws. Each is reachable by anyone who can (a) invoke the daemon, or (b) influence the file or query it is handed. There is no network listener in this binary; do not write "unauthenticated remote root". The lesson is the **file/argument-to-root-exec path** and the **public-identifier key**, not a magic remote exploit.

Build the Target

The target rebuilds reproducibly in a pinned `linux/arm64` container:

```
./firmware/build_target.sh      # -> ctfnode.stripped + ctfnode.unstripped
./ghidra/make_project.sh       # -> ghidra/proj/CTFNodeRE.gpr (headless analysis)
```

The build script also runs the stripped node natively in the container to prove it executes, and then prints the SHA-256 of both files.

Part 2: Your Assignment

Whenever a task asks you to **Document** findings or **answer questions**, write your answers in a single file named `TELESCREEN-Answers.md`. It will contain your identity record, function tables, defect analyses, the key-schedule write-up, your hardened design, and your written responses.

Task 1: Verify the Artifact (10 points)

1. Compute the SHA-256 of `firmware/ctfnode.stripped` and compare it to the instructor-issued value.
2. Record the ELF identity: class, machine, linkage, interpreter, and that it is stripped.
3. List the **dynamic imports** with `readelf -rW firmware/ctfnode.stripped | grep JUMP_SLOT` and state which ones are dangerous in this context.

Document: the hash comparison, the ELF identity table, and the import list with the dangerous ones flagged.

Task 2: Recover the Command Surface (15 points)

1. Find the program entry that glibc actually calls. Note the `__wrap_main` shim at `0x400874`, `main` at `0x400800`, and where `main` tail-calls.
2. Locate the dispatcher `ctf_dispatch` at `0x400c20` and list each subcommand string it compares against, in order.
3. Recover each subcommand's argument-count rule from the `cmp / ccmp` instructions.

Document: the entry chain (`_start -> __libc_start_main -> __wrap_main -> main -> ctf_dispatch`), the subcommand table with string addresses, and the argument rules.

Task 3: Name Every Function (20 points)

There are **ten application functions** in the binary. For each, give the address, the `FUN_` label Ghidra shows, the real name, and the **rule** that proves it:

- **R1** exact address match against `firmware/ctfnode.unstripped` (the answer key),
- **R2** a `.plt` stub -> `JUMP_SLOT` relocation -> import name,
- **R3** the `.plt` PLT to lazy resolver (`0x4006f0`),
- **R4** a phantom/overlapping function on alignment padding (`0x400adc`).

Also note every helper the compiler **inlined away** (there are five: one byte-fold helper and four `static` dispatcher helpers). A name you cannot find is a name the compiler removed - prove it.

Document: the address -> name -> rule table for all ten application functions, plus the inlined-helper list.

Task 4: Prove the Six Defects (30 points)

For each defect, give the **exact address** and the **instruction(s)** that implement it. A finding without the machine code is a guess.

defect	prove it with
B1	the <code>fopen</code> call, the <code>run=</code> word compare (<code>0x3d6e7572</code>), and the <code>system</code> call
B2	the <code>snprintf(..., "ping -c 1 %s", ...)</code> format string and the <code>system</code> call
B3	the <code>snprintf(..., "tar -xvzf %s -C /", ...)</code> format string and the <code>system</code> call
B4	the <code>strcmp(user, "admin")</code> and the empty-password test
B5	the <code>system("/bin/sh")</code> call and the dispatcher path that reaches it
B6	the reflected CRC-32 fold (<code>0xEDB88320</code>) and the 32-iteration key loop over the UID

Document: a per-defect table of address, function, the offending call, and a one-line exploit path; plus the honest threat-model note (local, not remote).

Task 5: Break the Weak Key Schedule (20 points)

1. Recover the exact derivation from `ctf_weak_key ( 0x4009b0 )` and `ctf_crc32_le ( 0x400960 )`. State the polynomial, the seed, the iteration count, and which byte is emitted.
2. Reproduce it in code. `scripts/weak_decrypt.py --uid SSAT-468547-FEEBD` is the reference and must print:

```
da506e04af00c6f40394d2cd2295bfc8682e8b9f9e9b844cea50c08d5f483141
```

3. Explain in 200 words why deriving a key from a public identifier is **not encryption**, and state the correct construction (secret exchange + KDF + AEAD).

Document: the derivation formula, the emitted bytes for the sample UID, your own reproduction, and the 200-word analysis.

Task 6: Demonstrate the Defects Locally (15 points)

Run the provided harnesses on the stripped binary (Docker `linux/arm64` on any host, or natively on aarch64 Linux):

```
python3 scripts/test_defects.py      # B1-B5 effects
python3 scripts/test_consistency.py  # the key equals the Python tool and the vector
```

Paste the output and explain what each `PASS` line proves. For **B6**, show that the binary's `key <uid>` output and the Python tool agree.

Document: the harness output and a one-line interpretation per check.

Task 7: Write the Hardened Replacement (15 points)

Describe - with code sketches - the hardened version of each defect. At minimum:

1. **B1:** stop sourcing config as root; run a fixed allow-list of actions, or parse and validate, never `system()`.
2. **B2:** replace the shell with `execve` and an **argument vector**; never build a command string.
3. **B3:** validate archive entry names and **refuse** `../`; extract into a staging directory, then move with least privilege.
4. **B4:** remove the default credential; require a real, provisioned secret.
5. **B5:** compile the debug shell out of production builds.
6. **B6:** replace the public-ID KDF with **X25519 ECDH + HKDF-SHA256** and **AES-256-GCM**, with a per-message nonce.

Document: the six fixes, your AEAD design (key source, cipher, nonce, integrity), and why each fix closes the corresponding hole.

Task 8: The Defensive Playbook (bonus, 10 points)

Write a one-page blue-team playbook: how to **detect** a TELESCREEN-class daemon in the field (process audit, config-write monitoring, outbound beacon analysis) and how to **harden** a deployed device (secure boot, signed updates, secret-key AEAD, read-only rootfs).

Deliverables Checklist

#	Deliverable	Format
1	Artifact identity: hash, ELF table, dangerous imports	Answers
2	Command surface: entry chain, subcommand table, argument rules	Answers
3	Function map: address -> name -> rule for all ten functions	Answers
4	Defect catalogue B1-B6 with instruction-level evidence	Answers
5	Crypto break: derivation, sample vector, reproduction, analysis	Answers
6	Harness output with per-check interpretation	Answers
7	Hardened replacement and AEAD design	Answers
8	Defensive playbook (bonus)	Answers
9	Full submission packaged as <code>lastname-firstname-TELESCREEN.zip</code>	ZIP

Grading

Task	Points	Criteria
1	10	Hash matches, ELF identity correct, dangerous imports flagged
2	15	Entry chain and subcommand table recovered correctly
3	20	Every application function named with a valid rule
4	30	All six defects proven with address + instruction evidence
5	20	Key schedule exact, sample reproduced, analysis sound
6	15	Harnesses run and results interpreted
7	15	Fixes correct and complete; AEAD design sound
8	10	Playbook is practical and detection-oriented
Total	135	(Task 8 is bonus; base 125)

Academic Integrity

By submitting this project, you certify that:

1. This is your own work.
2. You have not shared answers with other students.
3. You understand the ethical implications of surveillance-device research.
4. You will only use these skills for lawful purposes, on hardware you own, in an isolated lab.

"Freedom is the freedom to say that two plus two make four."

The skills you are demonstrating are the same ones used by security researchers, malware analysts - and by the ministries of real surveillance states. The fictional "TELESCREEN" is a stand-in for the very real class of cheap, backdoored, unsigned cameras and routers that sit in millions of homes today. The engineers who built the silicon we are studying are **teachers, not villains** - the craft is admirable; the surveillance is the crime. Always use your knowledge ethically, and only on hardware you are authorised to test.

Good luck, and happy (ethical) hacking.

Reference Material

Function Address Reference

address	Ghidra label	resolved name	role / defect
0x400800	FUN_00400800	main	tail-calls ctf_dispatch
0x400874	FUN_00400874	__wrap_main	linker --wrap shim -> main
0x400960	FUN_00400960	ctf_crc32_le	reflected CRC-32 (0xEDB88320)
0x4009b0	FUN_004009b0	ctf_weak_key	B6 weak key schedule
0x400a8c	FUN_00400a8c	ctf_login	B4 admin / empty password
0x400ae0	FUN_00400ae0	ctf_config_run	B1 config-sourced exec
0x400b64	FUN_00400b64	ctf_build_cmd	B2 ping -c 1 %s
0x400b80	FUN_00400b80	ctf_http_handle	B2 system(cmd)
0x400bc0	FUN_00400bc0	ctf_restore	B3 tar -xvzf %s -C /
0x400c00	FUN_00400c00	ctf_debug_shell	B5 system("/bin/sh")
0x400c0c	FUN_00400c0c	ctf_banner	prints the banner
0x400c20	FUN_00400c20	ctf_dispatch	subcommand dispatcher

Inlined Helpers (not present as functions)

source helper	inlined into
ctf_crc32_byte	ctf_crc32_le , ctf_weak_key
ctf_cmd	ctf_dispatch
ctf_print_key	ctf_dispatch
ctf_try_path	ctf_dispatch
ctf_try_misc	ctf_dispatch

String Table (.rodata)

address	string
0x400e68	admin
0x400e78	r (fopen mode)
0x400e80	run=
0x400e88	ping -c 1 %s
0x400e98	tar -xvzf %s -C /
0x400eb0	/bin/sh
0x400eb8	TELESCREEN node - the wall unit sees you
0x400ee8	config
0x400ef0	http
0x400ef8	restore
0x400f00	login
0x400f08	shell
0x400f10	key
0x400f18	%02x

Crypto Quick Reference

```
Weak (Ministry):  key = crc32_le_fold( public UID )    -> obfuscation, not encryption
Hardened (fix):  key = HKDF( X25519(priv,pub) )       -> AES-256-GCM
Nonce rule:      NEVER reuse a nonce under one key (GCM nonce reuse = forgery)
Integrity:       AEAD tag - forged frames die at the tag
```

Tools Required

- **Docker Desktop** (or Docker Engine) - runs the aarch64 target and the harnesses
- **JDK 21** and **Ghidra 12.1.3** - static analysis
- `readelf`, `aarch64-linux-gnu-objdump`, `aarch64-linux-gnu-nm`, `strings`
- Python 3 - the key tool, the harnesses, verification
- **No hardware required** for the analysis; the defects are demonstrated in the pinned container.

New to this? Install everything, step by step, for **Windows x64**, **Linux x64**, or **macOS arm64** in the companion course ([telescreen](#) → [docs/31-prerequisites-and-install.md](#)), then follow [docs/33-ghidra-nation-state-re.md](#).

Hints and Tips

1. **Trust the bytes, not the labels** - a stripped binary has no labels; the `.plt / .got.plt` relocations name the library calls.
2. `main` **is two hops away** - `__wrap_main` -> `main` -> `ctf_dispatch`.
3. **Inlining hides helpers** - `ctf_crc32_byte` appears twice, never by name.
4. **Find system first** - every `bl 0x400770` is a candidate defect.
5. **The `run=` compare is a 32-bit word** - `0x3d6e7572` is `r,u,n,=` in little-endian.
6. **The key is a reflected CRC-32** - polynomial `0xEDB88320`, **no** final XOR, low byte per round.
7. **State the threat model** - file/argument-to-root-exec, not network remote root.
8. **Document everything** - your analysis is the deliverable that survives the cell.